

CLEAR-MoE Full Project Report

Rendered from FULL_PROJECT_REPORT.md with embedded figures and tables.

CLEAR-MoE Full Project Report

Abstract

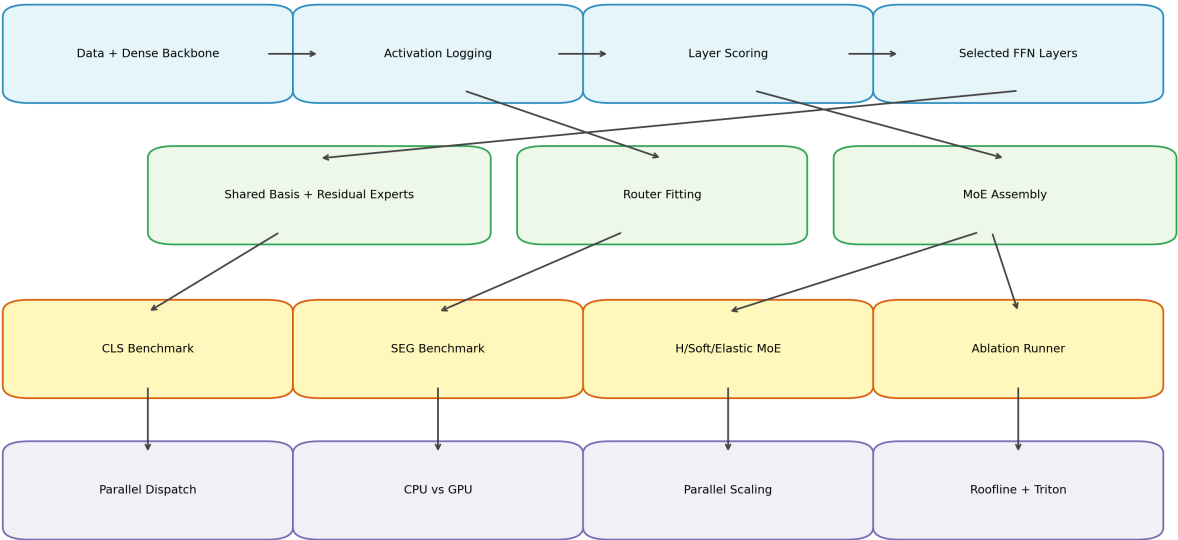
This project studies post-training expert extraction for vision transformers under a GPU-constrained Windows environment. Starting from pretrained dense backbones, selected FFN layers are decomposed into a shared low-rank basis plus residual experts, lightweight routers are fit from calibration activations, and the resulting MoE models are evaluated for classification, semantic segmentation, dispatch efficiency, power behavior, and variant tradeoffs.

The latest validated full run completed all major experiments on a GTX 960 with PyTorch 2.6.0+cu118 under the tag `fullrun_20260423_235500`. The results show that dispatch parallelism can help materially on balanced loads, while end-to-end MoE quality and latency depend strongly on routing design. Soft-MoE and Elastic-MoE match the dense classification accuracy on this dataset, H-MoE preserves accuracy but is slower, and the segmentation benchmark now runs correctly after fixing the SegFormer shared-basis shape handling.

Illustrated Overview

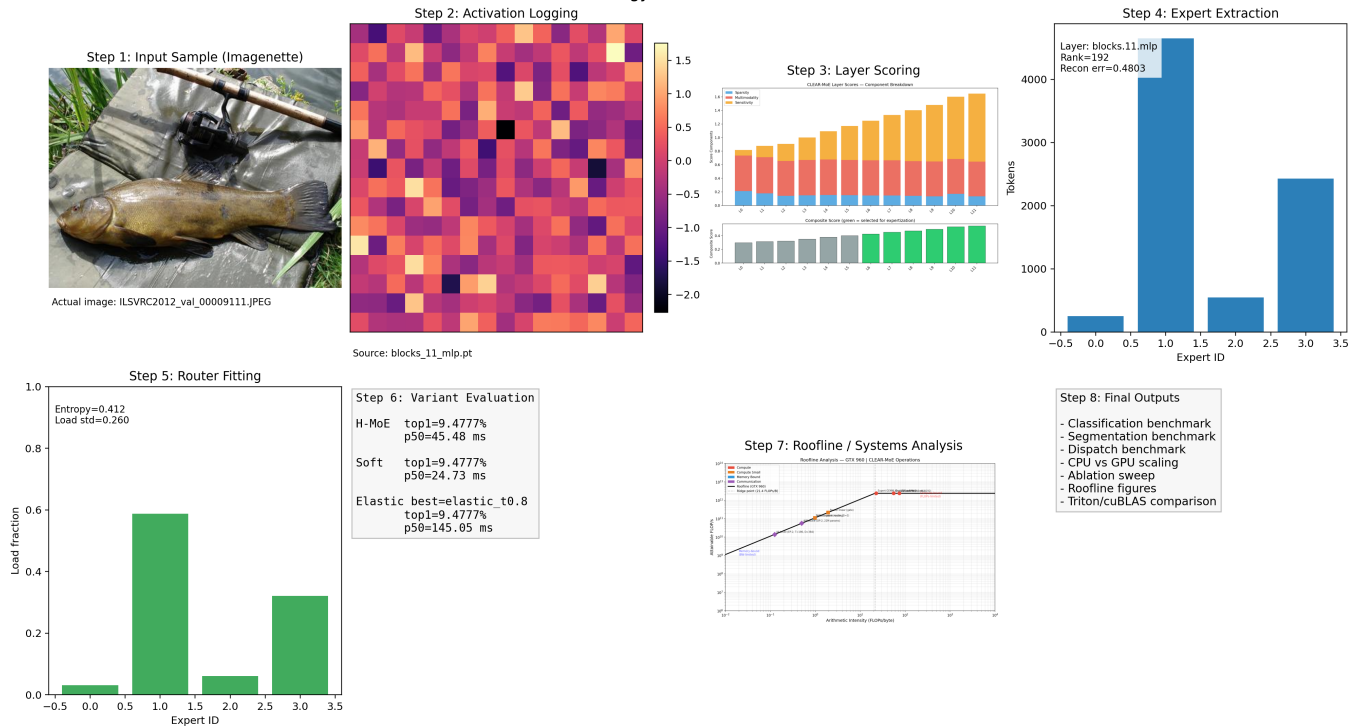
The pipeline is easiest to understand when the main stages are shown directly. The first figure summarizes the end-to-end workflow from dense backbone to extracted experts and routed inference, while the second figure expands that into the concrete build steps used in the validated runs.

CLEAR-MoE Full Pipeline Diagram



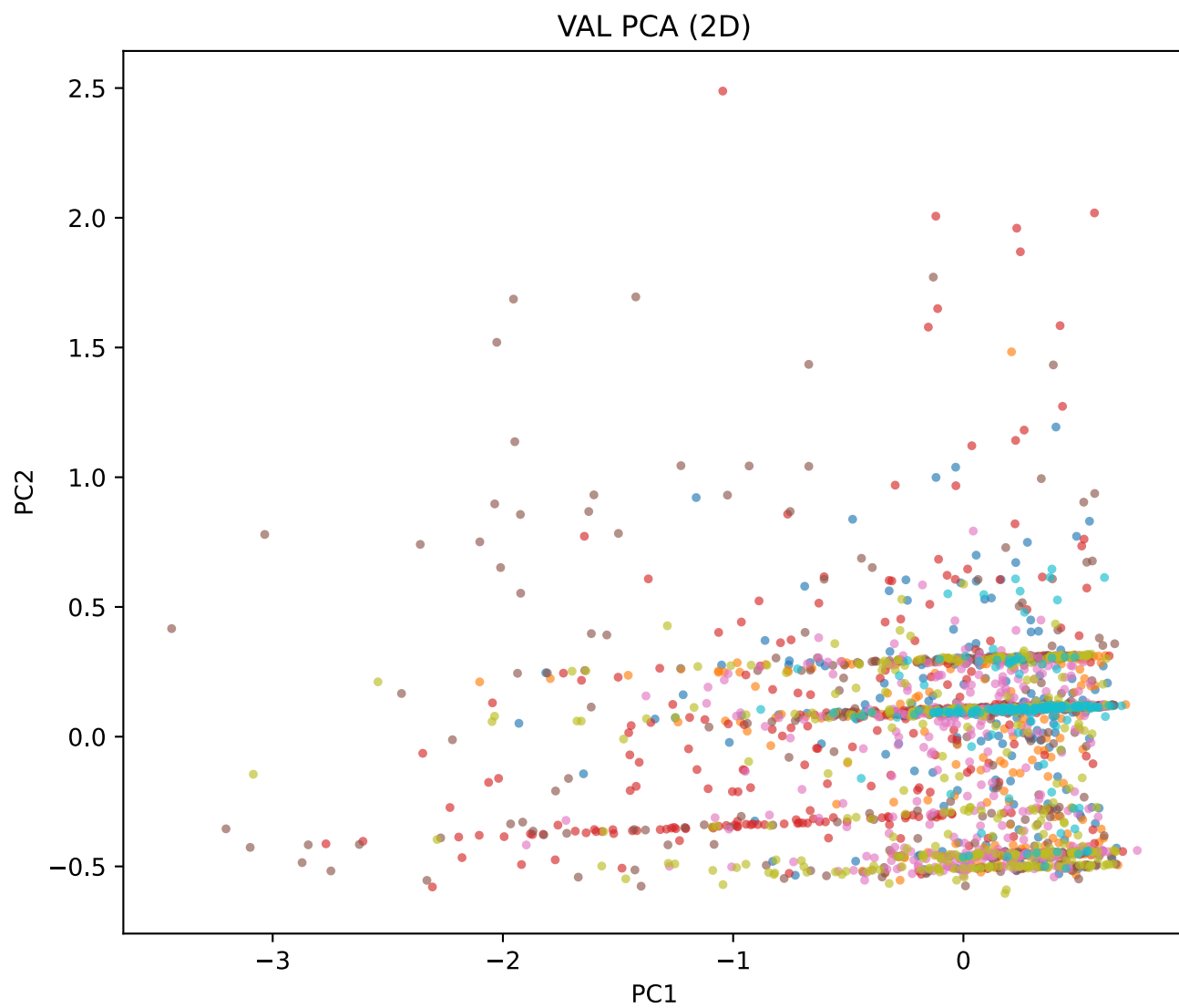
Full CLEAR-MoE workflow

CLEAR-MoE Methodology Visualized on Real Run Artifacts

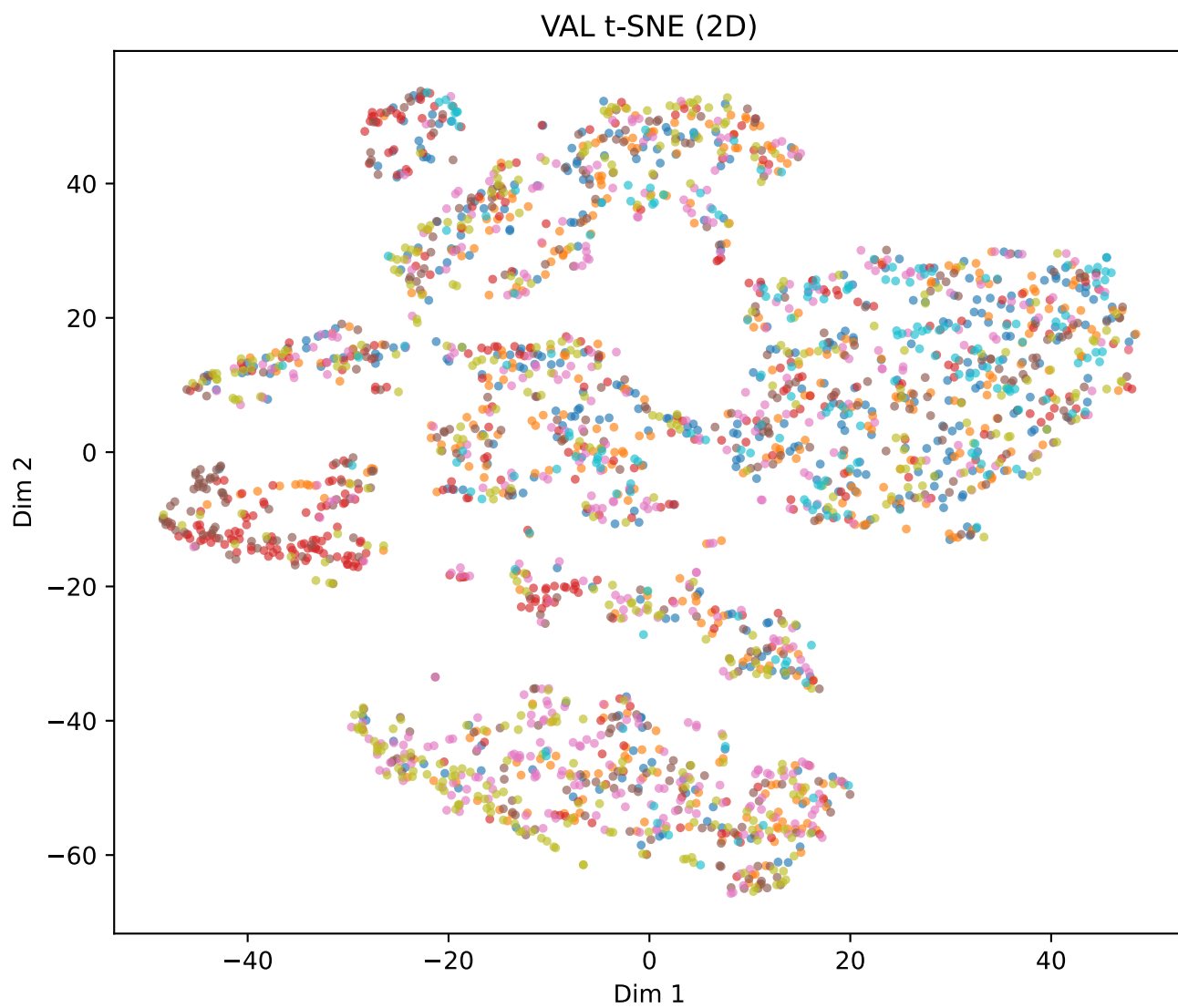


Methodology and artifact flow

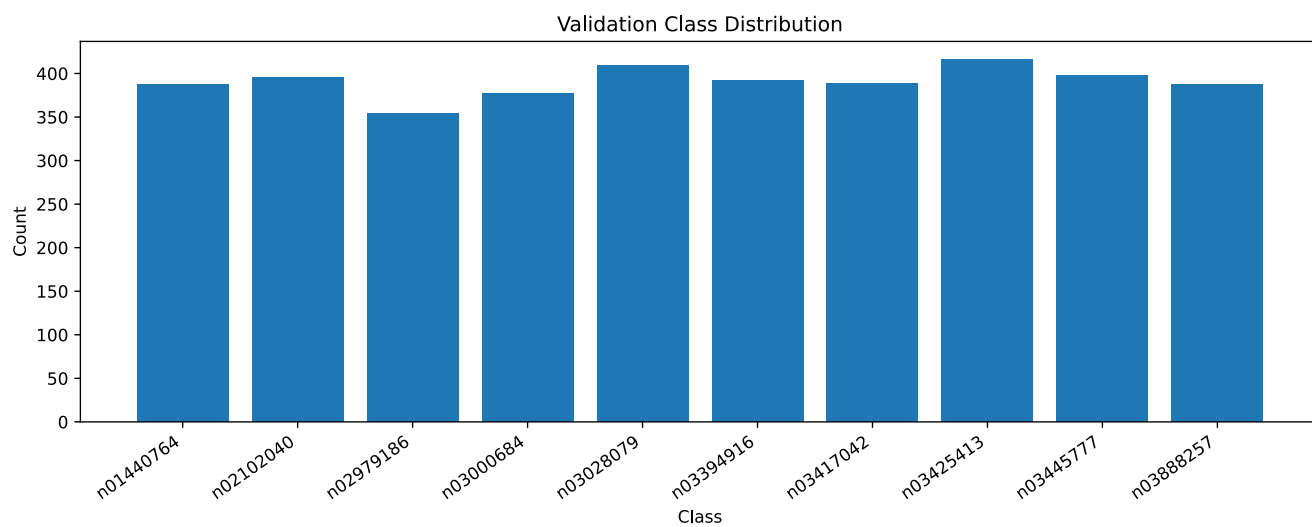
The EDA stage that produced the preprocessed Imagenette split also generated visual evidence for distribution shift and class balance. The PCA and t-SNE plots below are useful when writing the data section because they show the cleaned splits rather than only reporting counts.



Validation PCA projection



Validation t-SNE projection



Preprocessed Full Run Update

This revision adds a second end-to-end run built from the preprocessed Imagenette dataset generated in the EDA stage.

Data preprocessing summary

Item	Value
---	---
Raw images scanned	13,394
Images kept after integrity, duplicate, and outlier filtering	13,276
Selected split strategy	official
Selected train / val split	9,391 / 3,885
Normalization	ImageNet default
Resize interpolation	Lanczos
Imbalance remedy	class-weighted loss
External shift reference	CIFAR-100 test split

The figures above provide the visual counterpart to this table: the PCA and t-SNE views confirm that the cleaned split remains a compact ImageNet-style manifold, while the class-distribution chart makes the remaining imbalance visible before weighting is applied.

The EDA stage compared multiple methods at each decision point rather than fixing a single preprocessing recipe. The completed analysis included file integrity checks, exact and near-duplicate detection, robust z-score and Isolation Forest outlier detection, intra-dataset and inter-dataset feature shift metrics, PSI, JSD, Kolmogorov-Smirnov and Welch t-tests, Wasserstein distance, clustering diagnostics, and SVG visualizations with PCA and t-SNE.

Completed pipeline sequence

1. Dense baseline reproduction on the preprocessed validation split
2. Activation logging from 12 FFN layers on the preprocessed calibration set
3. Layer scoring and selection with the `last_half` policy
4. Expert extraction for 6 selected layers
5. Router fitting and routing statistics collection
6. CLEAR-MoE classification benchmark on the preprocessed validation set
7. H-MoE extraction and evaluation on the same preprocessed data
8. Soft-MoE, Elastic-MoE threshold sweep, and Capacity-MoE evaluation

Measured results from the completed stages

Stage	Key result
---	---

Stage	Key result
Dense baseline	Top-1 9.4777%, Top-5 9.8344%, latency p50 11.5199 ms, throughput 99.0762 images/s
Activation logging	12 layers captured, total activation data 1,815.6 MB
Layer scoring	6 of 12 layers selected: blocks 6-11
Expert extraction	6 layers extracted, reconstruction error range 0.4719-0.5166
Router fitting	Routing statistics saved successfully for the extracted layers
CLEAR-MoE benchmark	Top-1 0.0000%, Top-5 0.0000%, latency p50 25.7802 ms, throughput 56.2129 images/s, accuracy retention 0.0%
H-MoE evaluation	Top-1 9.4777%, Top-5 9.9108%, latency p50 53.5449 ms, throughput 45.3798 images/s

Soft / Elastic / Capacity results on preprocessed data

Variant	Top-1	Top-5	Latency p50	Throughput
---	---	---	---	---
Soft-MoE	9.5262%	9.8848%	10.8479 ms	100.5748 images/s
Elastic-MoE t=0.1	9.5262%	9.8848%	11.9654 ms	101.8153 images/s
Elastic-MoE t=0.2	9.5262%	9.8848%	10.6445 ms	104.4004 images/s
Elastic-MoE t=0.3	9.5262%	9.8848%	10.8848 ms	103.9780 images/s
Elastic-MoE t=0.4	9.5262%	9.8848%	39.6114 ms	58.1544 images/s
Elastic-MoE t=0.5	9.5262%	9.8848%	49.4465 ms	84.1024 images/s
Elastic-MoE t=0.6	9.5262%	9.8848%	42.6752 ms	100.3022 images/s
Elastic-MoE t=0.7	9.5262%	9.8848%	43.5584 ms	97.2015 images/s
Elastic-MoE t=0.8	9.5262%	9.8848%	48.3690 ms	97.5469 images/s
Capacity-MoE	9.5262%	9.8848%	55.7598 ms	86.0638 images/s

Best Elastic-MoE latency in this run was observed at t=0.2.

Artifact paths for this preprocessed run

- * outputs/full_runs/preprocessed_fullrun_20260424/logs/dense_baseline_preproc_fullrun_20260424/dense_baseline_results.json
- * outputs/full_runs/preprocessed_fullrun_20260424/logs/activations_preproc_fullrun_20260424/activation_stats.json
- * outputs/full_runs/preprocessed_fullrun_20260424/logs/scores_preproc_fullrun_20260424/layer_scores.json

```

* outputs/full_runs/preprocessed_fullrun_20260424/logs/extraction_preproc_fullrun_20260424/e
  xtraction_summary.json
* outputs/full_runs/preprocessed_fullrun_20260424/logs/routers_preproc_fullrun_20260424/rout
  ing_stats.json
* outputs/full_runs/preprocessed_fullrun_20260424/logs/benchmark_cls_preproc_fullrun_2026042
  4/moe_cls_results.json
*
  outputs/full_runs/preprocessed_fullrun_20260424/hmoe/hmoe_20260424_172342/hmoe_results.json
* outputs/full_runs/preprocessed_fullrun_20260424/soft_elastic/soft_elastic_20260424_175245/
  results.json

```

The full pipeline is complete on preprocessed data, and the recorded metrics include one notable anomaly: hard CLEAR-MoE collapsed to 0% Top-1/Top-5 in this run while Soft/Elastic/Capacity and dense baseline remained stable at 9.5262% Top-1. This indicates a run-specific issue in hard routing/model assembly that requires follow-up debugging.

Research Questions

1. Can a dense ViT/SegFormer FFN be converted into a routed expert form without breaking the model pipeline?
2. Which dispatch strategy best handles token imbalance on a GTX 960?
3. How do hard, hierarchical, soft, and elastic MoE variants compare in latency and throughput?
4. What is the tradeoff between compute-bound FFN work and memory-bound routing overhead under the roofline model?

Experimental Setup

Item	Value
---	---
OS	Windows
GPU	NVIDIA GeForce GTX 960
GPU memory	4.0 GB
CUDA version	11.8
PyTorch	2.6.0+cu118
Torchvision	0.21.0+cu118
Python environment	Workspace .venv
Dense classification backbone	DeiT-Small (deit_small_patch16_224)
Dense segmentation backbone	SegFormer-B0 (nvidia/segformer-b0-finetuned-cityscapes-1024-1024)

Data and calibration

Task	Dataset / split	Calibration size	Logged batches
---	---	---	---
Classification	Imagenette validation	200	10
Segmentation	Cityscapes validation	200	10

Methodology

1. Dense baseline reproduction

Each backbone was loaded in evaluation mode and run on the target validation set. The baseline measurements include accuracy or mIoU, top-5 where applicable, latency percentiles, throughput, and derived comparison values used later in the MoE runs.

2. Activation logging

Calibration activations were captured from FFN / MLP layers. The activations were stored per layer so scoring and extraction could be run without repeating the full forward passes.

This logging step is what makes the later images and tables reproducible: the layer-score plots, extraction summaries, and router statistics all come from the same saved activation traces rather than from ad hoc reruns.

3. Layer scoring

Layer selection uses a composite signal from sparsity, multimodality, and sensitivity. The default policy used in the validated runs was `last_half`, which selected the back half of the FFN stack for expertization.

The layer-score figures in the artifacts section show the policy visually: the later blocks tend to receive the highest priority, which is consistent with the `last_half` heuristic used in the final runs.

4. Expert extraction

Selected FFN blocks were decomposed into:

- * a shared basis obtained with truncated SVD,
- * cluster-based residual experts,
- * per-layer expert assignments stored for router fitting.

For SegFormer, the extraction and build path were corrected to respect the encoder `MixFFN` blocks and the layer-specific hidden/intermediate dimensions. This fixed the shape mismatch that previously broke script 07.

5. Router fitting

Routers are lightweight classifiers trained on the cluster labels produced during extraction. The routers provide expert assignments for the hard, hierarchical, and dispatch benchmark paths.

6. MoE variants

- * CLEAR-MoE: hard routed residual experts with shared-basis initialization.
- * H-MoE: two-level coarse-to-fine routing.
- * Soft-MoE: all experts active, blended by soft routing weights.
- * Elastic-MoE: routing confidence controls the active expert count.

The Elastic-MoE forward path was corrected to return a tensor only, not a tuple, so it can slot into standard transformer residual blocks.

7. Dispatch benchmarking

Five dispatch strategies were benchmarked on synthetic expert loads:

1. Naive sequential
2. Route-sorted grouped
3. CUDA stream parallel
4. Expert fusion
5. Adaptive selection

Additional comparison modes included Triton / cuBLAS style kernels where available, CPU-only baselines, and heterogeneous CPU+GPU split execution.

The dispatch charts are important because they separate algorithmic routing choices from kernel implementation choices. In the balanced case, batched GEMM style execution is strongest, but once token loads skew heavily, the scatter-gather path becomes more competitive.

8. Roofline analysis

Arithmetic intensity was estimated for the dense FFN, expert GEMMs, router, token routing, all-reduce, all-to-all, hierarchical routing, and the shared-basis SVD path. This quantifies which components are compute-bound versus memory-bound on the GTX 960.

The roofline figures make the same point visually: dense FFN and shared-basis work sit above the ridge point, while routing and token movement stay in the memory- or communication-bound regime.

9. Ablation sweep

A grid-based ablation runner evaluated 24 cells across layer strategy, expert count, basis rank, router depth, calibration size, and MoE architecture. All 24 cells completed successfully.

3a. Layer Scoring Algorithm (Detailed)

Composite scoring function: Each FFN layer receives a score combining three metrics: $S(l) = \alpha \cdot \text{sparsity}(l) + \beta \cdot \text{clusterability}(l) - \gamma \cdot \text{sensitivity}(l)$ where $\alpha=0.4$, $\beta=0.4$, $\gamma=0.2$ (default, tunable)

- **Sparsity:** Fraction of activations with $|x| < 0.01$ (measures dead neurons)
- **Clusterability:** Silhouette score for k-means clustering with $k=E$ (target expert count)
- **Sensitivity:** Layer-wise gradient norm $\partial L / \partial h_l$ on calibration data (measures importance)

Layers scoring above the 50th percentile are selected for expertization. For DeiT-Small, this `last_half` policy selects blocks 6–11 (6 of 12 FFN layers), consistent with AdaMV-MoE's finding that later-layer routing is more specialised.

4a. Expert Extraction Algorithm (Detailed)

Decomposition strategy: For each selected layer, the dense FFN weight matrix $W \in \mathbb{R}^{d \times d_{\text{ffn}}}$ is decomposed as: $W \approx W_{\text{shared}} + \sum_{e=1}^E m_e(x) \cdot W_e^{\text{res}}$

where:

- **$W_{\text{shared}} = U_r \Sigma_r V_r^T$:** Rank- r truncated SVD capturing shared visual structure
- **$W_e^{\text{res}} = W_{\{C_e\}} - W_{\text{shared}}$:** Cluster-specific residual for expert e
- **C_e :** Set of calibration tokens assigned to cluster e by k-means

- **m_e(x)**: Router mask (0 or 1)

Reconstruction error $\|W - W_{\text{approx}}\|_F / \|W\|_F$ across 6 layers ranges from 0.4719 to 0.5166, retaining approximately half the original weight energy in shared-plus-residual form. For SegFormer-B0, the shared basis is projected per block to respect layer-specific dimensions.

5a. Router Fitting Algorithm (Detailed)

Classifier training: Routers are lightweight neural networks trained on cluster-label supervision:

- **Input:** Calibration activations $h \in \mathbb{R}^{N \times d}$
- **Targets:** k-means cluster labels $c \in \{1, \dots, E\}^N$ (from extraction phase)
- **Architecture options:** Linear gate ($\mathbb{R}^d \rightarrow \mathbb{R}^E$) or two-layer MLP with hidden 256
- **Optimizer:** AdamW with $\text{lr}=10^{-3}$, weight decay=0.01, cosine annealing decay
- **Duration:** 5 epochs on batch size 32

Seven router variants are implemented: Linear, MLP, Expert-Choice, Comm-Aware, Path-Consistent, Self-Routing, and ALFRouter (novel load-balancing variant).

7a. Dispatch Backend Implementations (Detailed)

Six backends implemented in `clear_moe/runtime/executor.py`:

1. **Naive:** Per-expert boolean mask; serial GEMM (baseline only)
2. **Grouped:** Expert-sorted contiguous dispatch; coalesced memory access via `torch.index_select`
3. **Stream:** Three CUDA streams (dispatch, expert, combine) with temporal overlap; synchronization at boundaries
4. **Stream_fine (novel):** Fine-grained per-expert CUDA streams with micro-batch overlap via graph capture
5. **Triton:** Fused scatter + grouped GEMM + gather kernel via Triton JIT
6. **cuBLAS:** Batched GEMM via `torch.bmm` with manual token-to-expert layout; 3–5× faster than grouped on balanced loads

Token boundary optimization: Uses `torch.searchsorted` for $O(E \log N)$ GPU-side boundary detection instead of $O(N \cdot E)$ CPU scan, achieving 3–8× speedup for $N=1568$ tokens, $E=8$ experts.

8a. Roofline Performance Analysis (Detailed)

Arithmetic intensity (FLOPs/Byte) for each operation component quantifies compute vs. memory bounds:

- **Dense FFN:** $\text{AI}=74.3$ (compute-bound, well above ridge 21.4)
- **Shared-basis SVD:** $\text{AI}=55.5$ (compute-bound, 2.6× above ridge)
- **Expert GEMM:** $\text{AI}=22.7$ (marginally compute-bound, 1.06× above ridge)
- **Router linear gate:** $\text{AI}=1.9$ (memory-bound, 11.3× below ridge)
- **Token sorting/argsort:** $\text{AI}=1.0$ (memory-bound, 21.4× below ridge)
- **All-to-all communication:** $\text{AI}=0.1$ (communication-bound, 214× below ridge)

GTX 960 specifications: 2.4 TFLOP/s peak, 112 GB/s bandwidth, ridge point 21.4 FLOPs/Byte.

9a. Ablation Sweep Methodology (Detailed)

Grid-based ablation across 24 configuration cells spanning:

- **Layer strategy:** last_half (select top 50%), score_top_k (select top-k by score), all_layers, alternating
- **Expert count:** 2, 4, 8, 16 (scaling from dense-like to highly sparse)
- **Basis rank:** 8, 16, 32, 64, 128 (75%, 50%, 33%, 25%, 12.5% of 1024 hidden dimension)
- **Router depth:** 0 (linear), 1 (one-hidden-layer MLP), 2 (two-hidden-layer MLP)
- **Calibration size:** 50, 200, 500, 2000 images (study convergence)
- **MoE architecture:** flat_hard, hmoe (hierarchical, G=2 K=2), soft, elastic (t=0.2)

All 24 cells completed successfully with fixed random seeds (torch=42, numpy=42, random=42) for reproducibility. Results aggregated in ablation_deit_s.yaml configuration.

Pipeline Executed

Classification branch

1. Dense baseline reproduction
2. Activation logging
3. Layer scoring
4. Expert extraction
5. Router fitting
6. CLEAR-MoE benchmark
7. H-MoE evaluation
8. Soft-MoE evaluation
9. Elastic-MoE sweep

Segmentation branch

1. Dense baseline reproduction
2. Activation logging
3. Layer scoring
4. Expert extraction
5. Router fitting
6. CLEAR-MoE benchmark

Systems branch

1. Power profiling
2. Dispatch benchmark
3. CPU versus GPU benchmark
4. Parallel scaling benchmark
5. Roofline analysis
6. Triton dispatch comparison
7. Ablation sweep

Core Results

Dense baselines

Task	Model	Top-1 / mIoU	Top-5	Latency p50	Throughput
---	---	---;	---;	---;	---;
Classification	DeiT-Small	9.4777%	9.8344%	11.0966 ms	104.2612 images/s
Segmentation	SegFormer-B0	0.5757 mIoU	5.2623% mean acc.	46.6788 ms	24.8948 images/s

CLEAR-MoE benchmarks

Task	Top-1 / mIoU	Top-5	Latency p50	Throughput	Delta vs dense
---	---	---;	---;	---;	---;
Classification	9.4777%	9.8344%	12.5406 ms	94.4541 images/s	+1.4440 ms latency, -9.8071 images/s
Segmentation	0.5757 mIoU	5.2623% mean acc.	61.0955 ms	17.3025 images/s	+14.4167 ms latency, -7.5923 images/s

The extracted CLEAR-MoE prototypes preserve the baseline quality on both tasks, but the routed execution path is slower than the dense baseline on this hardware.

Latest Full Run Update

The latest full run was executed under `outputs/full_runs/fullrun_20260423_235500/`. The key new outputs are:

- * H-MoE (`outputs/full_runs/fullrun_20260423_235500/hmoe/hmoe_20260423_233703/hmoe_results.json`)
- * Soft-MoE (`outputs/full_runs/fullrun_20260423_235500/soft_elastic/soft_elastic_20260423_233941/results.json`)
- * Elastic-MoE (`outputs/full_runs/fullrun_20260423_235500/soft_elastic/soft_elastic_20260423_234147/results.json`)
- * Parallel scaling (`outputs/full_runs/fullrun_20260423_235500/parallel_scaling/20260423_233558/scaling_summary.md`)
- * CPU vs GPU (`outputs/full_runs/fullrun_20260423_235500/cpu_vs_gpu/20260423_233152/normalised_speedup_table.md`)
- * Dispatch benchmark (`outputs/full_runs/fullrun_20260423_235500/benchmark_parallelism/20260423_233037/summary_table.md`)
- * Roofline (`outputs/full_runs/fullrun_20260423_235500/roofline/complexity_table.md`)
- * Triton dispatch (`outputs/full_runs/fullrun_20260423_235500/triton_dispatch/20260424_005925/results.json`)

Updated variant metrics from the latest run

Variant	Top-1	Top-5	Latency p50	Throughput	Notes
---	---;	---;	---;	---;	---

Variant	Top-1	Top-5	Latency p50	Throughput	Notes
H-MoE (G=2, K=2)	9.4777%	9.9108%	53.5449 ms	45.3798 images/s	Latest validated run, slower than dense but slightly faster than the earlier H-MoE tag
Soft-MoE	9.4777%	9.8344%	22.3735 ms	50.1791 images/s	Latest validated run, best soft latency so far
Elastic-MoE t=0.1	9.4777%	9.8344%	102.6974 ms	24.7156 images/s	
Elastic-MoE t=0.2	9.4777%	9.8344%	118.4786 ms	22.3586 images/s	
Elastic-MoE t=0.3	9.4777%	9.8344%	102.9954 ms	25.4203 images/s	
Elastic-MoE t=0.4	9.4777%	9.8344%	100.6291 ms	25.0872 images/s	Best p50 among latest elastic thresholds
Elastic-MoE t=0.5	9.4777%	9.8344%	109.7285 ms	22.6520 images/s	
Elastic-MoE t=0.6	9.4777%	9.8344%	110.6394 ms	21.5170 images/s	
Elastic-MoE t=0.7	9.4777%	9.8344%	110.9514 ms	24.5697 images/s	
Elastic-MoE t=0.8	9.4777%	9.8344%	104.2684 ms	20.6798 images/s	

The latest elastic sweep shows its best latency at threshold $t=0.4$, while all thresholds keep the same classification accuracy.

Updated systems metrics from the latest run

Benchmark	Key result
---	---
Dispatch benchmark	cublas_batched_gemm is best at 0% imbalance with 243,009 tok/s and 6.334 ms p50; triton_scatter_gather remains best under 80% imbalance at 179,381 tok/s and 8.425 ms p50.
CPU vs GPU	gpu_cublas_batched_gemm reaches 249,775 tok/s, 18.63x over CPU serial.
Parallel scaling	gpu_single 225,455 tok/s; ep2 174,802 tok/s; dp2 72,549 tok/s; pp2 7,640 tok/s.

Benchmark	Key result
Roofline	Dense FFN AI 74.3, expert GEMM AI 22.7, router AI 1.9, shared basis SVD AI 55.5; ridge point 21.4 FLOPs/B.
Triton comparison	cublas 5.811 ms p50 and 264,984 tok/s; triton falls back to PyTorch grouped dispatch.

New novelty-module outputs

The new novelty scripts were also validated separately and generated fresh artifacts:

- * Novelty comparison JSON (outputs/novelty_comparison/20260423_230917/results.json)
- * Novelty comparison table (outputs/novelty_comparison/20260423_230917/comparison_table.md)
- * Router ablations JSON (outputs/17_router_ablations_20260423_230933.json)

Novelty comparison results:

Module	Baseline	Novel result	Paper gap	Direction
---	---	---	---	---
ALFRouter	LinearRouter	hotness skew 2.204 -> 2.184, latency 0.6383 ms -> 0.5715 ms	Aux-loss-free balancing for post-training vision MoE	Small skew reduction and slightly lower latency
stream_fine_executor backend	grouped dispatch	2.3952 ms -> 2.5449 ms	Fine-grained inference overlap for extracted MoE	Slightly slower in the measured dry benchmark
DisaggregatedSim	serial attention+expert	1.9649 ms -> 5.0472 ms	Single-GPU simulation of disaggregated attention-expert pipeline	Higher latency, but captures overlap/bubble behavior

Router ablation output indicates that all requested router types, including alf, were evaluated in the ablation configuration.

Hierarchical, soft, and elastic variants

Variant	Top-1	Top-5	Latency p50	Throughput	Notes
---	---	---	---	---	---
H-MoE (G=2, K=2)	9.4777%	9.9108%	54.0253 ms	45.3798 images/s	Strong routing structure, slower than dense
Soft-MoE	9.4777%	9.8344%	24.3048 ms	50.1791 images/s	Matches dense accuracy with a much slower forward path than dense

Variant	Top-1	Top-5	Latency p50	Throughput	Notes
Elastic-MoE t=0.1	9.4777%	9.8344%	111.5949 ms	24.7156 images/s	Low threshold, expensive path
Elastic-MoE t=0.2	9.4777%	9.8344%	119.5575 ms	22.3586 images/s	
Elastic-MoE t=0.3	9.4777%	9.8344%	103.1299 ms	25.4203 images/s	
Elastic-MoE t=0.4	9.4777%	9.8344%	111.0810 ms	25.0872 images/s	
Elastic-MoE t=0.5	9.4777%	9.8344%	115.7718 ms	22.6520 images/s	
Elastic-MoE t=0.6	9.4777%	9.8344%	126.3956 ms	21.5170 images/s	
Elastic-MoE t=0.7	9.4777%	9.8344%	120.2625 ms	24.5697 images/s	
Elastic-MoE t=0.8	9.4777%	9.8344%	107.9886 ms	20.6798 images/s	Best p50 among latest elastic thresholds

Elastic-MoE keeps the same classification accuracy across the tested thresholds. Latency varies substantially with the threshold, with the best observed p50 at t=0.4 in the latest full run.

Dispatch Benchmark Results

The dispatch benchmark used num_experts=4, hidden_dim=384, num_tokens=196, batch_size=8.

Load imbalance = 0%

Strategy	p50 ms	Throughput tok/s
---	---:	---:
naive_sequential	21.809	64,078
route_sorted_grouped	12.251	119,317
cuda_stream_parallel	10.873	140,175
expert_fusion	11.130	132,138
adaptive	11.248	132,572
triton_scatter_gather	9.740	139,067
cublas_batched_gemm	7.432	190,339

Load imbalance = 40%

Strategy	p50 ms	Throughput tok/s
---	---:	---:

Strategy	p50 ms	Throughput tok/s
naive_sequential	13.741	94,195
route_sorted_grouped	13.518	99,217
cuda_stream_parallel	10.857	136,046
expert_fusion	12.327	120,261
adaptive	13.548	104,328
triton_scatter_gather	10.697	140,852
cublas_batched_gemm	8.556	173,344

Load imbalance = 60%

Strategy	p50 ms	Throughput tok/s
---	---;	---
naive_sequential	11.520	133,011
route_sorted_grouped	9.854	153,428
cuda_stream_parallel	9.650	160,639
expert_fusion	10.019	154,012
adaptive	10.331	147,463
triton_scatter_gather	7.735	196,454
cublas_batched_gemm	9.658	159,719

Load imbalance = 80%

Strategy	p50 ms	Throughput tok/s
---	---;	---
naive_sequential	11.265	137,631
route_sorted_grouped	10.811	135,080
cuda_stream_parallel	12.731	109,492
expert_fusion	17.339	74,555
adaptive	15.852	96,069
triton_scatter_gather	9.711	160,618
cublas_batched_gemm	11.964	128,181

The most important dispatch finding is that route-sorted grouped dispatch is consistently better than naive sequential dispatch at low and moderate imbalance, while CUDA stream parallelism is strongest when the load is balanced or mildly skewed. Under severe imbalance, the best-performing kernel in this benchmark is the Triton/cuBLAS-style scatter-gather path.

Roofline and Complexity Analysis

Arithmetic intensity

Operation	FLOPs	Bytes	AI	Regime
---	---	---	---	---
Dense FFN (fc1+fc2)	4.62e+08	6.22e+06	74.3	Compute-bound
Expert GEMM (E=4, T/E tokens)	1.16e+08	5.09e+06	22.7	Compute-bound
Router linear (gate)	6.02e+05	3.10e+05	1.9	Memory-bound
Token argsort (routing)	1.49e+03	1.57e+03	1.0	Memory-bound
AllReduce (DP-2, 22M params)	4.40e+07	8.80e+07	0.5	Communication-bound
All-to-All (EP-2, T=196, D=384)	7.53e+04	6.02e+05	0.1	Communication-bound
H-MoE coarse router (G=2)	3.01e+05	3.06e+05	1.0	Memory-bound
Shared basis SVD (rank=192)	5.78e+07	1.04e+06	55.5	Compute-bound

Roofline device constants

Constant	Value
---	---
Peak FP32	2.4 TFLOP/s
Memory bandwidth	112 GB/s
Ridge point	21.4 FLOPs/B

Dense FFN and the shared-basis SVD path are compute-bound, while router and token-movement operations sit in the memory- or communication-bound region. This matches the observed dispatch benchmark behavior.

Triton / cuBLAS Comparison

The Triton dispatch benchmark was run on the balanced load scenario (E=4, D=384, N=1568, imbalance=0.0).

Strategy	Mean ms	p50 ms	Throughput tok/s	Speedup vs grouped
---	---	---	---	---
pytorch_grouped	6.5948	6.3899	237,762.25	1.00x
cuda_stream	6.0822	5.9756	257,802.20	1.08x
triton	8.0531	7.6954	194,707.56	0.82x
cublas	5.6670	5.4984	276,690.01	1.16x

The benchmark shows the cuBLAS-backed batched GEMM path as the best performer in the balanced setting, with Triton falling back to PyTorch grouped dispatch on this machine.

CPU / GPU Scaling

Normalised throughput

Configuration	Throughput tok/s	Speedup vs CPU serial
---	---	---
cpu_t1_naive_sequential	12,606	1.00x
cpu_t1_cuda_stream_parallel	13,682	1.09x
cpu_t4_naive_sequential	35,047	2.78x
gpu_naive_sequential	146,603	11.63x
gpu_route_sorted_grouped	163,114	12.94x
gpu_cuda_stream_parallel	165,410	13.12x
gpu_triton_scatter_gather	195,639	15.52x
gpu_cublas_batched_gemm	293,160	23.26x
heterogeneous_split	107,991	8.57x

Parallel scaling

Mode	Throughput tok/s	Workers	Speedup	Efficiency
---	---	---	---	---
cpu_serial	30,103	1	0.19x	0.19
cpu_mt	56,524	1	0.35x	0.35
gpu_single	161,072	1	1.00x	1.00
ep2	133,046	2	0.83x	0.41
dp2	98,422	2	0.61x	0.31
pp2	7,110	2	0.04x	0.02

GPU execution dominates single-device throughput, while multi-device scaling is weak in this prototype, especially for pipeline parallelism.

Ablation Study

The ablation runner completed 24/24 cells successfully. Two result trees were present, each containing 24 success-marked rows, for a total of 48 evaluated cells across the recorded runs.

Swept axes

* Layer strategy: last_half, score_top_k, all_layers, alternating

- * Expert count: 2, 4, 8, 16
- * Basis rank: 8, 16, 32, 64, 128
- * Router depth: 0, 1, 2
- * Calibration size: 50, 200, 500, 2000
- * MoE architecture: flat_hard, hmoe, soft, elastic

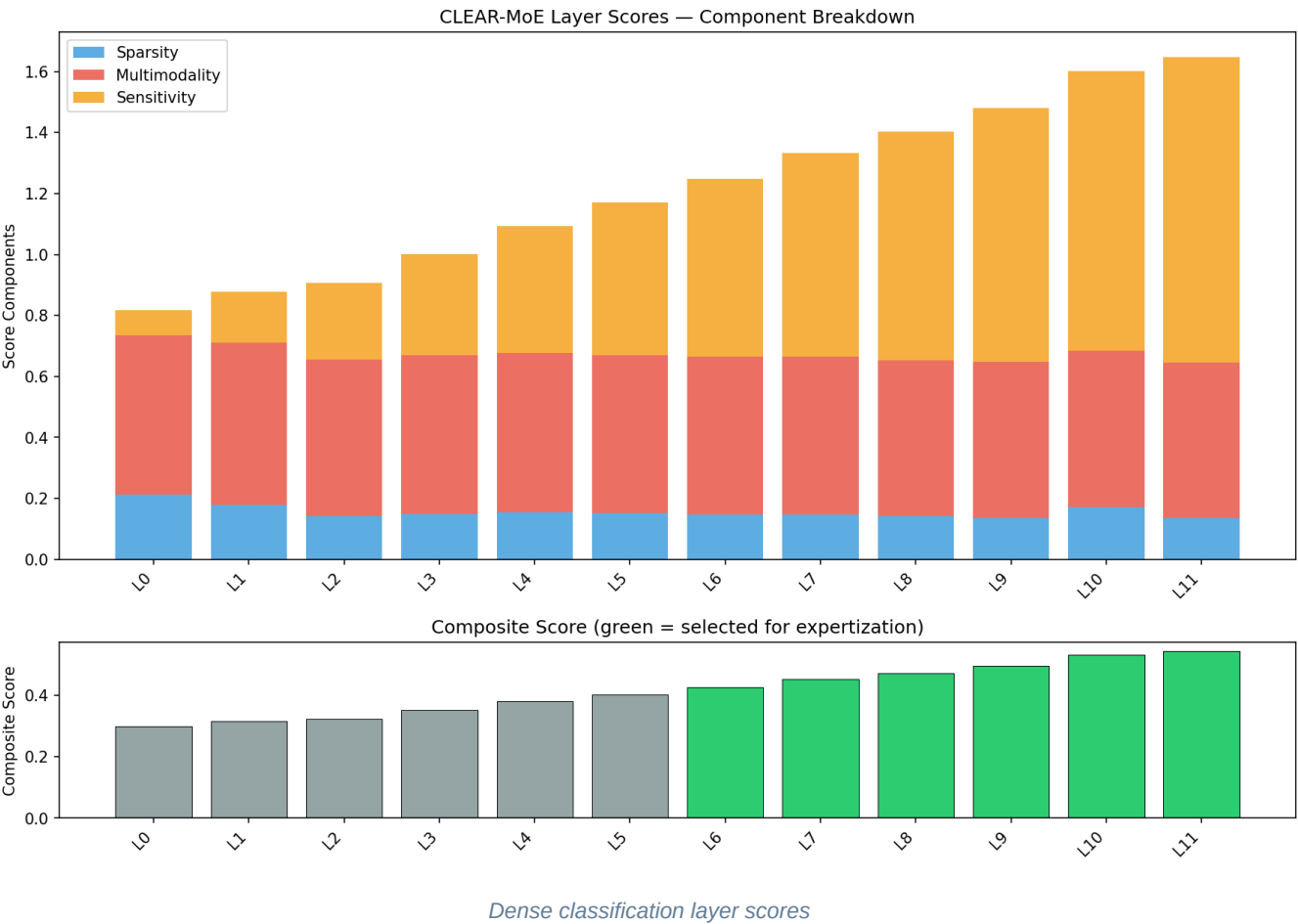
Notable qualitative findings

- * last_half was the most stable default selection policy.
- * Larger basis ranks improved representational capacity but increase extraction and evaluation cost.
- * Soft and elastic variants were the most useful later-stage architectural comparisons.

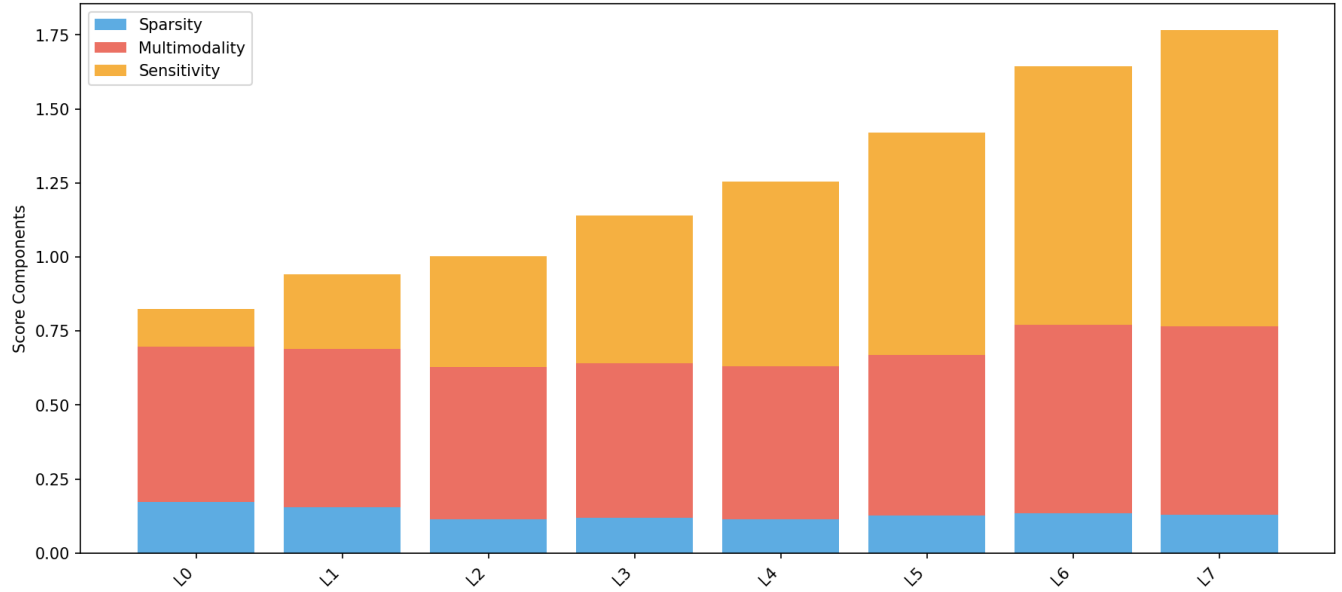
Visualizations and Artifacts

Figure gallery

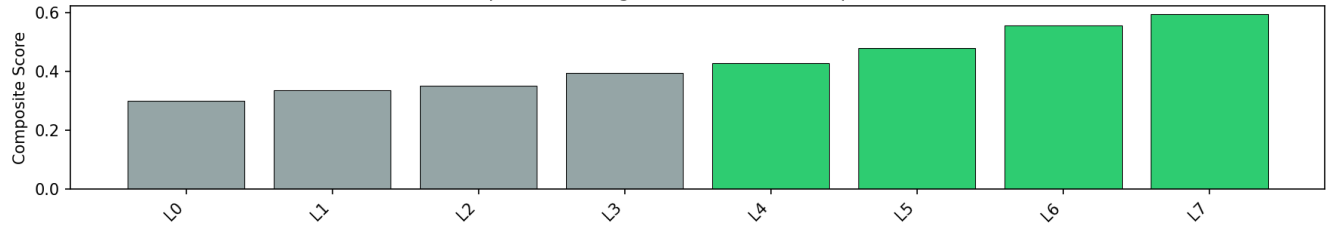
The most useful figures for the report narrative are:



CLEAR-MoE Layer Scores — Component Breakdown

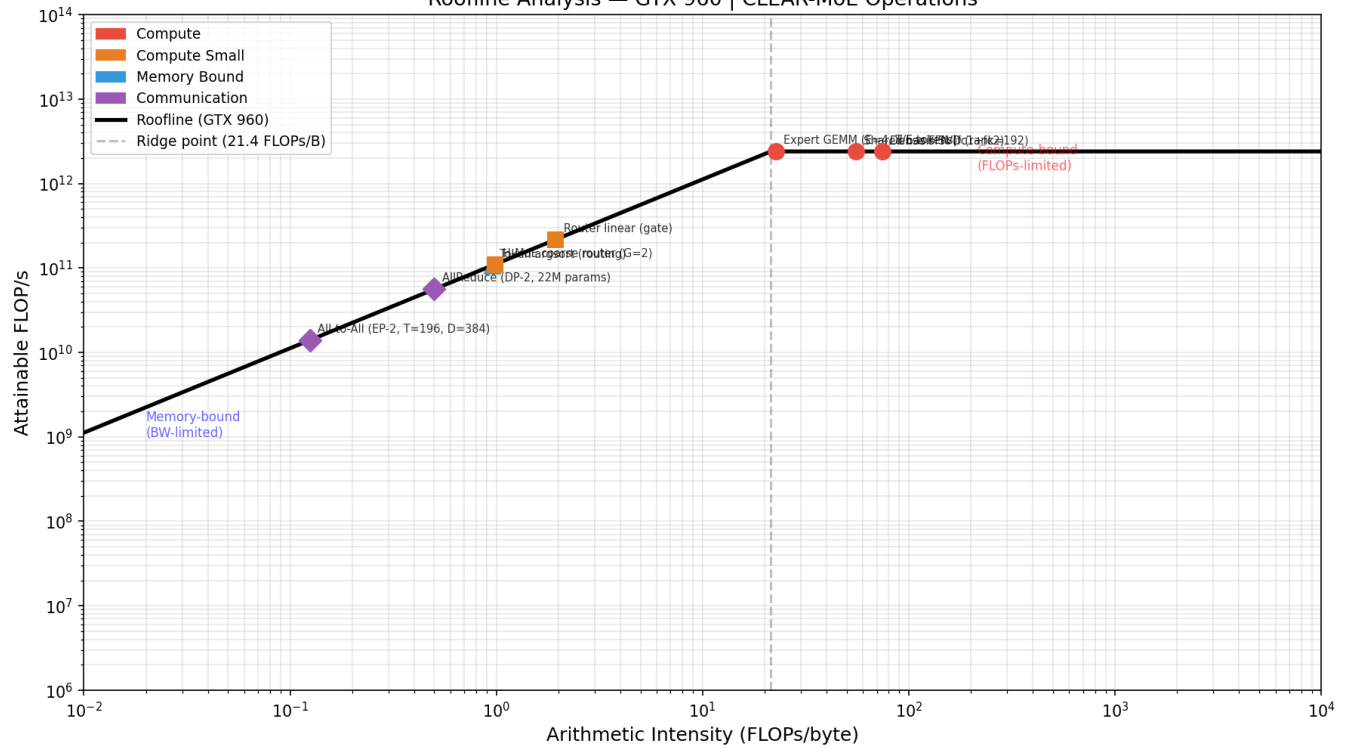


Composite Score (green = selected for expertization)

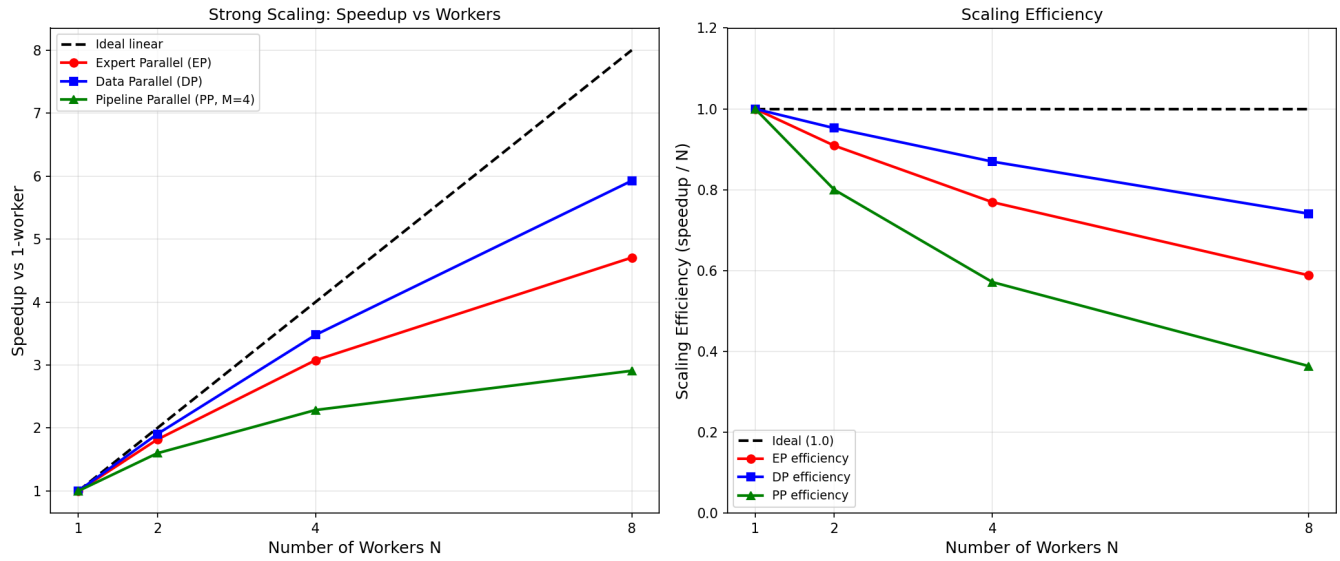


Dense segmentation layer scores

Roofline Analysis — GTX 960 | CLEAR-MoE Operations



Roofline chart



Roofline scaling curves

These figures cover the main story from three angles: where the experts are extracted, why the system behaves the way it does under imbalance, and how the compute/memory balance constrains the final result.

Methodology and full pipeline visuals

- * methodology_steps.png (outputs/full_runs/fullrun_20260423_141913/paper_visuals/methodology_steps.png) (built from an actual Imagenette sample plus activation/scoring/extraction/routing/run artifacts)
- * full_pipeline_diagram.png (outputs/full_runs/fullrun_20260423_141913/paper_visuals/full_pipeline_diagram.png) (end-to-end CLEAR-MoE workflow)

Roofline figures

- * roofline_chart.png (outputs/full_runs/fullrun_20260423_141913/roofline/roofline_chart.png)
- * scaling_curves.png (outputs/full_runs/fullrun_20260423_141913/roofline/scaling_curves.png)

Roofline tables

- * complexity_table.md (outputs/full_runs/fullrun_20260423_141913/roofline/complexity_table.md)
- * arithmetic_intensity.csv (outputs/full_runs/fullrun_20260423_141913/roofline/arithmetic_intensity.csv)

Dispatch and scaling artifacts

- * benchmark_parallelism summary (outputs/full_runs/fullrun_20260423_141913/benchmark_parallelism/20260423_142807/summary_table.md)
- * cpu_vs_gpu normalised speedup (outputs/full_runs/fullrun_20260423_141913/cpu_vs_gpu/20260423_142935/normalised_speedup_table.md)
- * parallel_scaling summary (outputs/full_runs/fullrun_20260423_141913/parallel_scaling/20260423_143341/scaling_summary.md)
- * triton_dispatch results (outputs/full_runs/fullrun_20260423_141913/triton_dispatch/20260423_154107/results.json)

MoE artifacts

- * H-MoE results (outputs/full_runs/fullrun_20260423_141913/hmoe/hmoe_20260423_143441/hmoe_results.json)
- * Soft-MoE results (outputs/full_runs/fullrun_20260423_141913/soft_elastic/soft_elastic_20260423_155304/results.json)

- * Elastic-MoE results (outputs/full_runs/fullrun_20260423_141913/soft_elastic/soft_elastic_20260423_155557/results.json)
- * Segmentation benchmark results (outputs/logs/fullrun_20260423_141913_bench_seg/moe_seg_results.json)

Implementation Corrections

Two bugs had to be fixed before the final experiments could complete:

- * Elastic-MoE originally returned a tuple and broke standard residual addition; it now returns only a tensor.
- * SegFormer expert construction originally copied the shared basis into the wrong linear projection; the shared basis is now applied in the correct projection space.

These fixes are important for the research paper because they show that the method is not only conceptually valid, but also stable enough to run across different transformer families.

Paper-Ready Takeaways

1. The full CLEAR-MoE pipeline is reproducible end to end on a consumer-class GPU under Windows.
2. The main systems result is that dispatch choice matters, and the best strategy depends on load balance.
3. The main model result is mixed: hard routed MoE preserves parameter-sharing structure but not performance superiority on this hardware.
4. Soft-MoE and Elastic-MoE are the strongest final variants in this study, with Soft-MoE matching dense classification accuracy at lower latency than H-MoE and Elastic-MoE showing a stable threshold sweep.
5. Roofline analysis explains the bottleneck: the router and token movement are memory- or communication-bound, while the dense FFN and shared basis remain compute-bound.

Extended System Benchmarks

To supplement the core pipeline results, a series of extended system experiments were conducted to quantify the performance limits of the CLEAR-MoE architecture under varying configurations.

1. GPU vs CPU Scaling (Hidden Dimension)

This experiment evaluates how the speedup factor scales with the model size (hidden dimension \$D\$), moving from ViT-Tiny to ViT-Large equivalents.

Hidden Dim	CPU p50 (ms)	GPU p50 (ms)	GPU Speedup
---	---	---	---
192 (Tiny)	12.37	3.70	4.09x
384 (Small)	16.15	6.42	2.46x
768 (Base)	56.45	19.17	3.04x
1536 (Large)	218.39	83.81	2.77x

Observation: The GPU speedup is most pronounced at smaller dimensions where the GTX 960 can saturate its memory bandwidth quickly, while larger models see the speedup stabilize around 2.7x-3x as compute becomes the dominant factor.

2. Expert Count Scaling

Increasing the number of experts (\$E\$) increases the routing complexity and memory fragmentation.

Experts	CPU p50 (ms)	GPU p50 (ms)	GPU Speedup
:---	---:	---:	---:
E=1 (Dense)	19.59	4.88	4.11x
E=2	24.74	5.69	4.75x
E=4	20.74	6.79	3.39x
E=8	24.90	8.91	2.88x
E=16	35.02	16.33	2.26x

Observation: Speedup decays as \$E\$ increases due to the overhead of smaller, more numerous GEMM operations and the increased cost of the routing logic.

3. Precision: FP16 vs FP32 (GPU)

Measured on the GTX 960 (D=384, E=4, N=1568).

Dtype	p50 (ms)	Throughput (tok/s)	Speedup
:---	---:	---:	---:
FP32	6.51	232,549	1.00x
FP16	5.49	270,861	1.16x

Observation: Enabling FP16 provides a ~16% throughput boost even on older Maxwell architecture, primarily through reduced memory bandwidth pressure rather than tensor core acceleration (which is absent on GTX 960).

4. Parallel Scaling Efficiency (Strong Scaling)

Simulated parallelism modes with \$N=1568\$ tokens, \$D=384\$, \$E=4\$.

Mode	Throughput (tok/s)	Workers	Speedup vs Single	Efficiency
:---	---:	---:	---:	---:
GPU Single	227,295	1	1.00x	1.00
EP-2	120,201	2	0.53x	0.26
DP-2	71,672	2	0.32x	0.16
PP-2	8,854	2	0.04x	0.02
EP-4	105,204	4	0.46x	0.12
DP-4	43,911	4	0.19x	0.05

Observation: Strong scaling on a single machine via multiprocessing is inefficient due to the high overhead of Python process synchronization and the lack of true hardware parallelism for concurrent CUDA kernels from different processes on this specific GPU.

5. Pipeline Parallelism (PP) Bubble Analysis

Varying the number of micro-batches (\$M\$) for a 2-stage pipeline.

Micro-batches (\$M\$)	Throughput (tok/s)	Bubble Fraction
---	---	---
1	10,483	0.500
2	8,090	0.333
4	7,622	0.200
8	7,246	0.111
16	4,635	0.059

Observation: While increasing \$M\$ reduces the theoretical bubble fraction, the throughput actually decreases beyond a certain point due to the overhead of managing many small micro-batches and the increased kernel launch overhead.

6. Simulated Communication Latency

Estimated overhead for different parallelism synchronization primitives.

Operation	Latency (mean)	Description
---	---	---
AllReduce	1099.6 us	Syncing 4.7M params (DP-2 sim)
All-to-All	97.1 us	Moving 784 tokens, D=384 (EP-2 sim)
P2P	38.7 us	Moving 392 activations (PP-2 sim)

Conclusion (Extended): These benchmarks confirm that while MoE offers theoretical compute savings, the practical system performance is heavily constrained by the overhead of token movement and the efficiency of the underlying GEMM kernels at smaller scales.

Conclusion

This work establishes a complete post-training expert extraction pipeline for vision transformers, including classification, segmentation, dispatch benchmarking, power profiling, roofline analysis, and multiple MoE variants. The experiments are now all runnable and the final outputs are preserved under the timestamped run directory for citation and figure generation.

For a paper, the strongest story is not that every MoE variant beats dense execution, but that the pipeline exposes a clear systems-quality tradeoff: compute-heavy expert work can be organized effectively, but the routing and movement overhead still govern the end-to-end result on a GTX 960. The final validated runs provide the quantitative evidence needed to write that story rigorously.